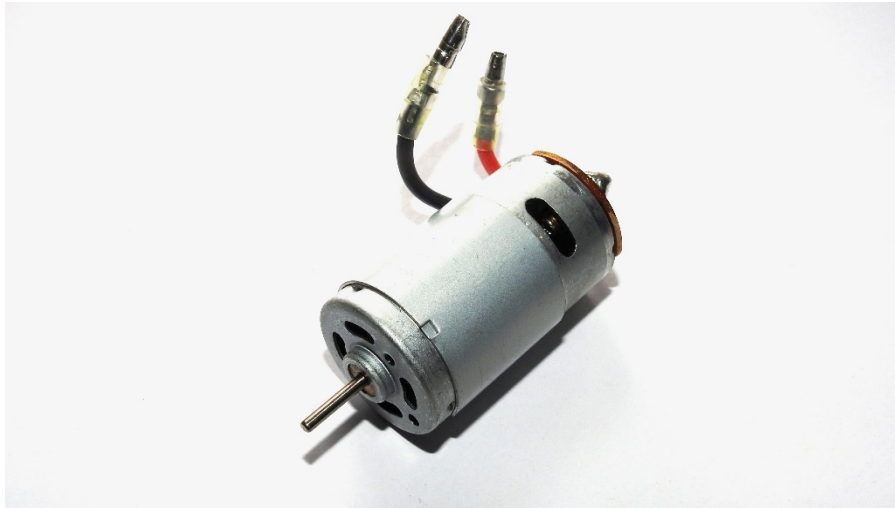


SENSORES, ATUADORES E ARMAZENAMENTO

TEMA 1 — PROJETO MOTOR CC

O motor CC pode ser comprado em qualquer loja de componentes eletrônicos ou ser retirado de um brinquedo, de um drive de CD de um desktop antigo, por exemplo. Na Figura 1, você pode ver um desses motores.

Figura 1 — Motor CC



Créditos: Blitzstock/Shutterstock.

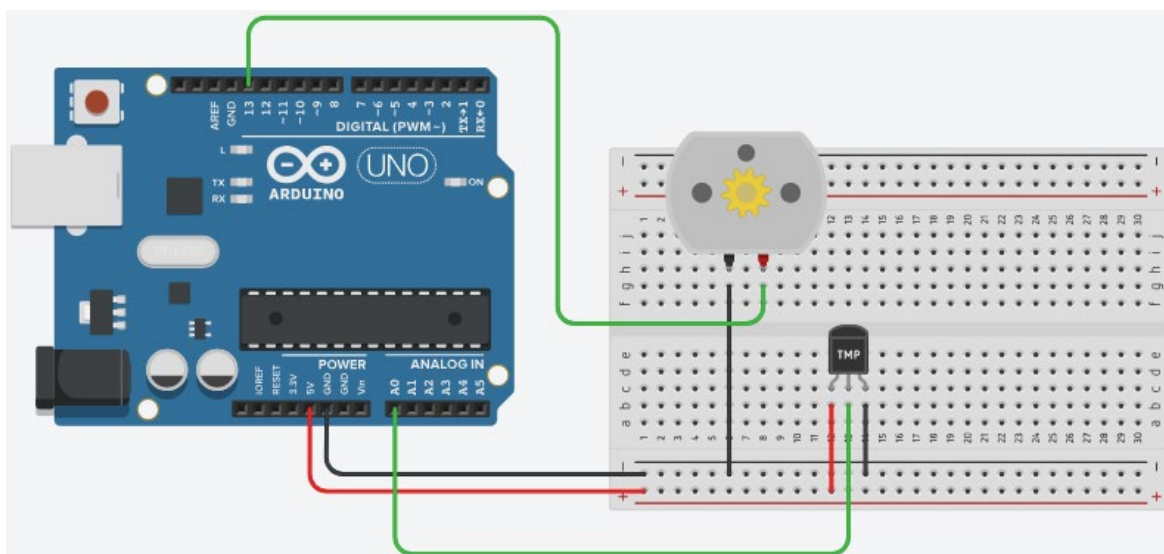
Neste projeto, também utilizaremos um sensor de temperatura para que o motor ligue ou desligue, dependendo da temperatura do ambiente.

Com essa simulação, é possível implementar um projeto que liga ou desliga um ventilador, por exemplo, dependendo da temperatura externa. Sempre que for manusear o Arduino, é importante que ele não esteja ligado a nenhuma fonte de energia (USB ou Jack), assim não terá problemas que podem queimá-lo. Para esse projeto, serão necessários:

- Arduino
- Sensor LM35
- Protoboard
- Jumpers
- Motor CC

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 2 representa as ligações necessárias.

Figura 2 — Representação da montagem do Projeto motor CC



Na Figura 2, vemos que o motor CC é conectado a uma porta digital (pino 13), e o sensor de temperatura a uma porta analógica (pino A0). É importante ter atenção às portas a que eles foram conectados, pois elas serão programadas no software. Então, após a montagem do hardware, é preciso programar, isto é, criar o software.

Figura 3 — Representação do sketch do Projeto motor CC

```
1 // Projeto Motor CC
2
3 int temp = 0; // inicializando a variável temp
4
5 void setup()
6 {
7     pinMode(A0, INPUT);
8     pinMode(13, OUTPUT);
9 }
10
11 void loop()
12 {
13     temp = analogRead(A0); // leitura da temperatura externa
14     if (temp > 30) { // decisão de ligar ou desligar o motor
15         digitalWrite(13, HIGH);
16     } else {
17         digitalWrite(13, LOW);
18     }
19     delay(100);
20 }
```

A variável “temp”, declarada na linha 3, está sendo inicializada com o valor 0. A inicialização de uma variável é importante, pois evita o chamado de “lixo

de memória”, ou seja, que algum valor seja atribuído para ela automaticamente. Então, o pino A0, utilizado para o sensor de temperatura, vai capturar (INPUT) um sinal dentro de um range (pegar a temperatura). Como essa temperatura é variável dentro de um intervalo, é utilizada uma porta analógica. O motor, conectado ao pino digital 13 (OUTPUT), pode apenas ser ligado ou desligado.

Dentro da função `loop()`, na linha 13, a variável “temp” recebe o valor que foi lido analogicamente no sensor que está plugado na porta analógica A0 (definida na linha 7). Já sabendo qual é o valor lido pelo sensor (linha 13), na linha 14 existe um comando de decisão, que pode ser traduzido como: se o valor que foi captado e está armazenado na variável “temp” for maior que 30, então ligue o que está conectado à porta digital 13 (o motor), se não (valor armazenado na variável “temp” menor ou igual a 30), desligue o que estiver conectado ao pino 13.

A função `delay()` serve para informar em quanto tempo será repetido o processo, sendo feita uma nova medição de temperatura.

Assim como o sensor de temperatura foi utilizado para ligar ou desligar um motor CC pequeno, ele também pode ligar ou desligar qualquer motor CC, independentemente do seu tamanho, pois o comando passado pelo Arduino é basicamente uma chave que libera ou não a alimentação.

TEMA 2 — PROJETO MOTOR CC: ALTERANDO A DIREÇÃO

O motor CC apresenta algumas particularidades, como a possibilidade de inverter sua rotação ou controlar a velocidade de rotação. Para inverter a rotação de um motor CC, é preciso mudar as fases para que ele funcione para lados diferentes, no sentido horário e no anti-horário.

A inversão de rotação é muito comum nos motores elétricos: os vidros elétricos dos carros a utilizam para abrir e fechar, assim como as parafusadeiras, que lançam mão dessa propriedade para parafusar e desparafusar.

O projeto agora consiste em utilizar um potenciômetro¹ para inverter a rotação de um motor CC. Para isso, serão necessários:

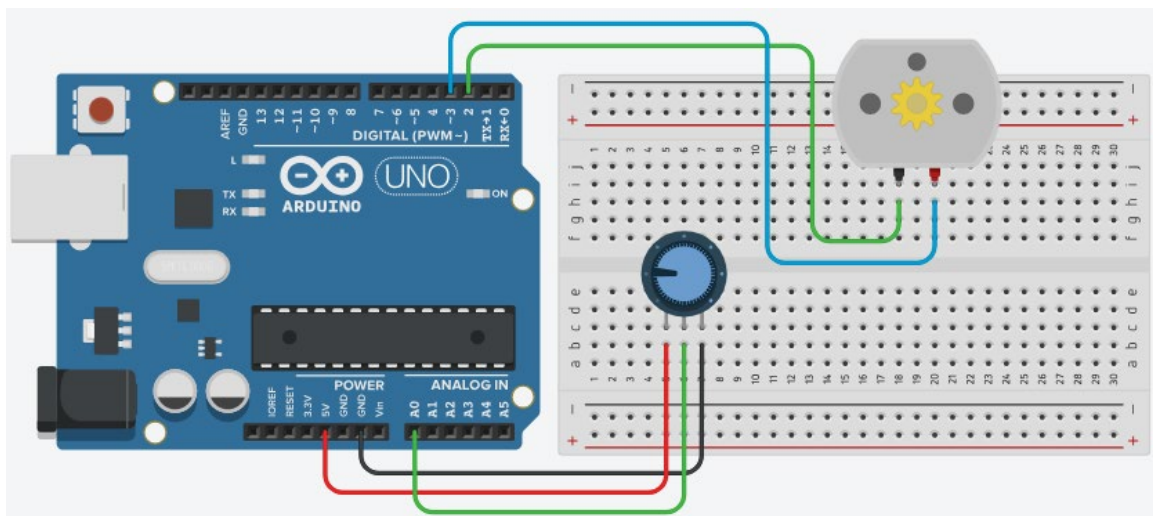
- Arduino
- Potenciômetro

¹ Um potenciômetro é um tipo especial de resistor de três terminais cuja resistência pode ser ajustada por meio mecânico, girando ou deslizando um eixo móvel, formando assim um divisor de tensão ajustável (Reis, 2018).

- Protoboard
- Jumpers
- Motor CC

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 4 mostra as ligações necessárias para o projeto.

Figura 4 — Representação da montagem do Projeto motor CC: alterando a direção



Na Figura 4, o motor CC é conectado a duas portas digitais: porta 2, com o jumper verde, e porta 3, com o jumper azul. O potenciômetro tem três pinos, os quais são conectados à alimentação (jumper vermelho), à porta analógica A0 (jumper verde) e ao GND (jumper preto). Como o potenciômetro mede valores diferentes dentro de uma grandeza, é preciso conectá-lo a uma porta analógica. Uma vez montado o hardware, é preciso criar o software.

Figura 5 — Representação do sketch do Projeto motor CC: alterando a direção

```
1 // Projeto Motor CC - Alterando a Direção
2
3 float valor_pot;
4 void setup()
5 {
6     pinMode(2, OUTPUT);
7     pinMode(3, OUTPUT);
8     Serial.begin(9600);
9 }
10
11 void loop()
12 {
13     valor_pot = (float)analogRead(A0) * (10.00 / 1023.00);
14
15     if (valor_pot >= 0 && valor_pot <= 5)
16     {
17         digitalWrite(2, HIGH);
18         digitalWrite(3, LOW);
19     }
20     if (valor_pot > 5 && valor_pot <= 10)
21     {
22         digitalWrite(2, LOW);
23         digitalWrite(3, HIGH);
24     }
25     Serial.println(valor_pot);
26 }
```

Na linha 3, foi declarada a variável “valor_pot” do tipo float. Nas linhas 6 e 7, é informado que as duas portas às quais o motor foi plugado são de saída (OUTPUT). Como o valor lido pelo potenciômetro vai ser impresso, a comunicação serial foi inicializada.

A leitura de um potenciômetro varia de 0 até 1.023. Quando o valor lido for 0, ele está recebendo 0v de alimentação e, quando o valor lido for 1.023, a alimentação é de 5v.

Voltando à Figura 5, na linha 13, a variável “valor_pot” recebe o valor lido pela porta A0 e o multiplica por (10.00 / 1.023). Esse cálculo serve para dividir o valor da leitura, em uma escala de 0 a 10. Agora que já é conhecido um valor de leitura entre 0 e 10, o programa determina duas condições:

- Na linha 15, é testado se o valor da variável “valor_pot” é maior ou igual a 0 e menor ou igual a 5, então é ligada a fase plugada no pino 2 e desligada a fase plugada no pino 3, girando no sentido horário.

-
- Na linha 20 é testado se o valor da variável “valor_pot” é maior que 5 e menor ou igual a 10, então é desligada a fase plugada no pino 2 e ligada a fase plugada no pino 3, girando no sentido anti-horário.

Na linha 20, é impresso o valor lido e convertido do potenciômetro no serial monitor. Note que, nas duas decisões, a condição é composta por duas sentenças utilizando o conectivo de adição “e”. Isso significa que, para a condição ser verdadeira, as duas sentenças devem ser verdadeiras.

TEMA 3 — PROJETO MOTOR CC: CONTROLANDO A VELOCIDADE

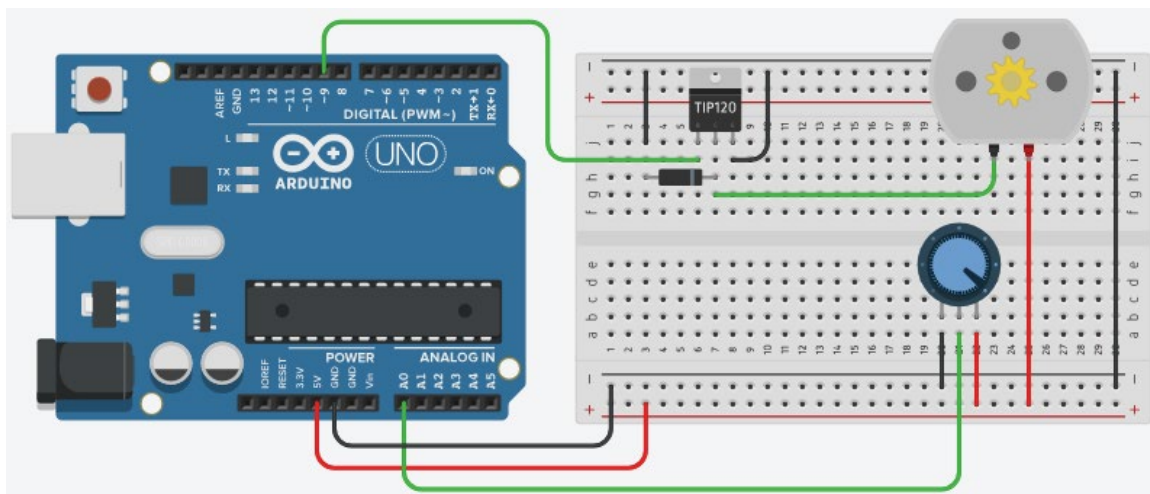
Os motores CC permitem o controle de velocidade de rotação. Um exemplo prático é uma furadeira, cujo motor gira mais rápido quanto mais o gatilho (botão) for pressionado.

O projeto consiste em utilizar um potenciômetro para regular a velocidade de rotação do motor. Quanto maior for o valor lido pelo potenciômetro, maior será a rotação. Para esse projeto, será necessário:

- Arduino
- Potenciômetro
- Protoboard
- Jumpers
- Motor CC
- Diodo
- Transistor TIP120

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 6 mostra as ligações necessárias.

Figura 6 — Representação da montagem do Projeto motor CC: controlando a velocidade



Na Figura 6, além do motor e do potenciômetro, foram utilizados outros dois componentes: o diodo e o transistor.

Um diodo é um componente eletrônico que deixa que a corrente elétrica passe somente em um sentido, da mesma forma que uma válvula que permite à água fluir apenas em um sentido. Observe a Figura 7, que ilustra um diodo.

Figura 7 — Representação de um diodo



Créditos: Serorion/Shutterstock.

Uma das funções do diodo é converter a tensão alternada em contínua, atuando como um retificador. Já o transistor é um componente eletrônico que amplifica pequenos sinais, podendo ainda atuar como uma chave liga/desliga, de acordo com seus sinais de entrada.

Agora que os dois novos componentes foram apresentados, passaremos para a montagem do projeto. Na parte direita da Figura 6, existe um jumper na trilha do GND, que fornece o aterramento para o diodo e o transistor. Repare que o motor CC é ligado ao transistor, e que o transistor é ligado na porta digital 9. Como o projeto é apenas para o controle de velocidade, um pino do motor vai no transistor e o outro na alimentação. O potenciômetro que vai regular a velocidade do motor está plugado na porta analógica A0. Depois da montagem do hardware, é preciso criar o software.

Figura 8 — Representação do Projeto motor CC: controlando a velocidade

```
1 // Projeto Motor CC - Controlando a Velocidade
2
3 // Pino analógico onde o potenciômetro está plugado
4 int const potenciometro = 0;
5
6 // Pino digital onde o TIP120 (transistor) está plugado
7 int const tip = 9;
8
9 int valPotenciometro = 0;
10
11 void setup() {
12     pinMode(tip, OUTPUT);
13 }
14
15 void loop() {
16     // Lendo o valor do potenciômetro (0 até 1023) e dividindo por 4
17     // para obter um valor entre 0 e 255 para a base do TIP120.
18     valPotenciometro = analogRead(potenciometro) / 4;
19
20     analogWrite(tip, valPotenciometro);
21 }
22 }
```

Na linha 4, foi declarada a constante “potenciômetro”, à qual foi atribuído o valor 0, simbolizando que o potenciômetro está plugado na porta analógica A0. Na linha 7, a constante “tip” foi definida com o valor 9, significando que o transistor está plugado no pino digital 9. A variável “valPotenciômetro” foi declarada do tipo inteiro e inicializada com o valor 0, na linha 9.

A constante “tip” é definida como saída (OUTPUT), na linha 12. Dentro da execução do programa, na linha 19, a variável “valPotenciômetro” recebe a leitura analógica do potenciômetro. Como o transistor trabalha com valores de 0 a 255, a leitura do potenciômetro (0 a 1.023) foi dividida por 4.

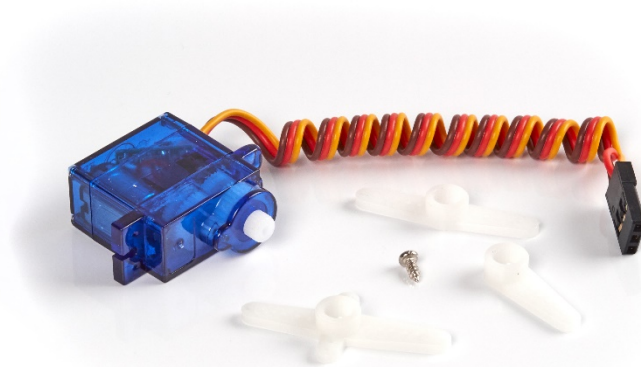
A linha 21 é a responsável por mandar a quantidade de energia para a porta 9 (tip). A função analogWrite passa dois parâmetros: a constante tip (porta 9)

e a variável valPotenciômetro (quantidade de energia). Quanto mais energia, maior é a velocidade do motor, e quanto menos energia, menor é a velocidade do motor.

TEMA 4 — PROJETO SERVOMOTOR

O servomotor é um motor diferente do CC, pois, ao receber um comando, o servo gira até o grau que foi determinado, sem ter o giro contínuo (CC). Para a utilização com o Arduino, o recomendado é o micro servo. O micro servo possui todas as características de funcionamento de um servomotor, mas com tamanho reduzido. Observe a Figura 9.

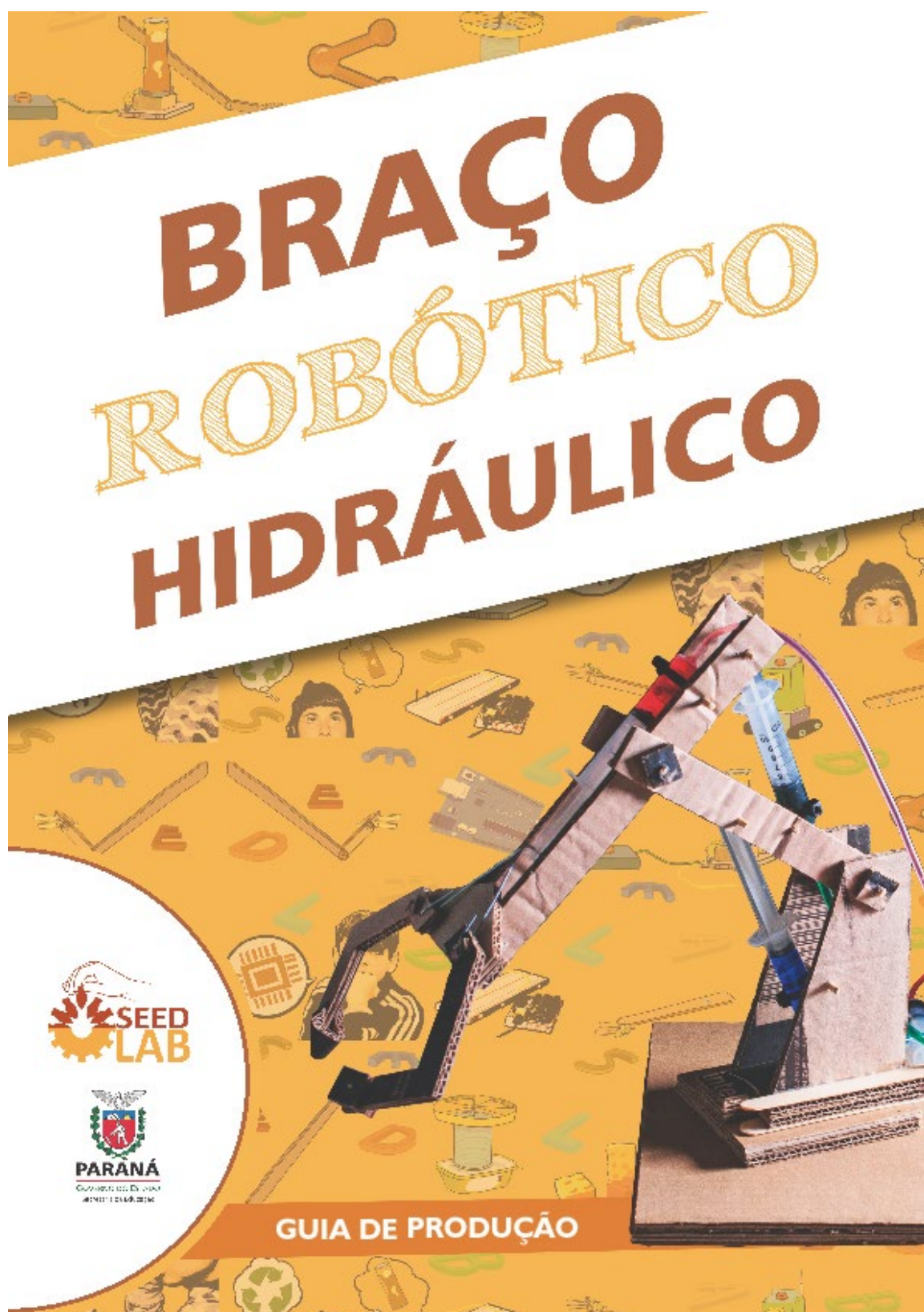
Figura 9 — Representação de um micro servo



Créditos: Teodor Costachioiu/Shutterstock.

Pela sua precisão de posicionamento e velocidade, os servos são muito utilizados na robótica. Observe a Figura 10, ela mostra um braço robótico que utiliza quatro servos para se movimentar.

Figura 10 — Braço robótico com quatro servos



Fonte: Guia de produção braço robótico hidráulico/SEED Lab/Governo do Estado do Paraná.

Para o projeto da Figura 10, o servomotor é o mais indicado. Analisando somente a garra, quando ela é fechada e segura alguma coisa, não pode mais se

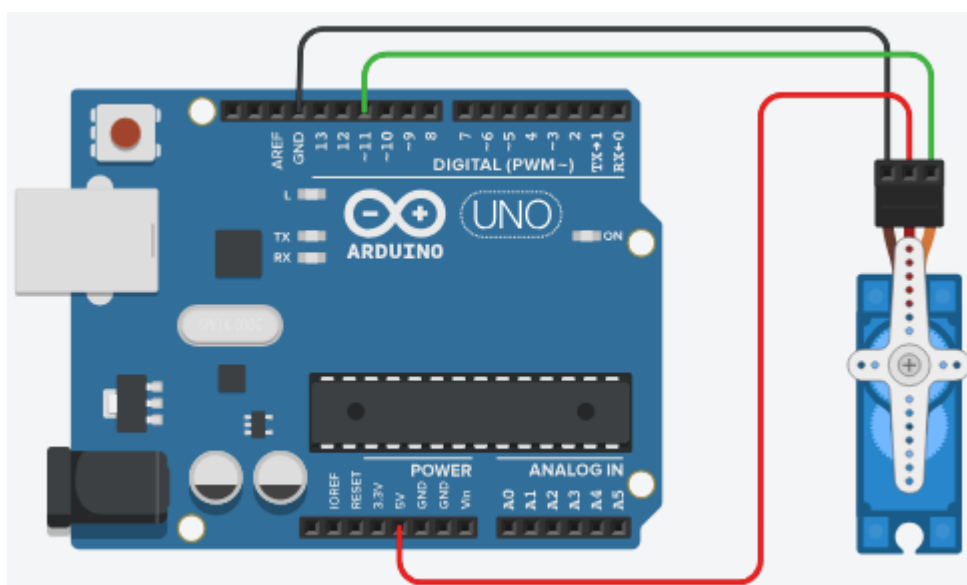
abrir, ou seja, a posição do motor não pode ser alterada. Essa é uma característica do servomotor.

Esse projeto consiste, então, na programação de um servomotor sem a interferência posterior do usuário. O servo vai executar uma tarefa repetitiva, sem nenhum botão ou potenciômetro para alterar seu funcionamento. Esse projeto demandará o seguinte:

- Arduino
- Jumpers
- Servomotor

Primeiro passo faremos a montagem do hardware do projeto. A Figura 11 mostra quais são as ligações necessárias.

Figura 11 — Representação da montagem do Projeto servomotor



A conexão do servomotor é feita com três jumpers (dois para alimentação e um para receber o sinal de controle, determinando a posição). Na Figura 11, vemos que não foi utilizado um protoboard para esse projeto. O primeiro pino foi conectado ao GND (jumper preto), o segundo pino, à alimentação (jumper vermelho) e o terceiro pino, à porta digital 11 (jumper verde). Após a montagem do hardware, é preciso programar.

Figura 12 — Representação do Projeto servomotor

```
1 //Projeto Servo Motor
2
3 #include <Servo.h>
4
5 Servo meu_servo; // Cria um objeto servo
6 int pos = 0; // Variável para armazenar a posição do meu_servo
7
8 void setup()
9 {
10     meu_servo.attach(11); // meu_servo conectado ao pino digital 11
11 }
12
13 void loop()
14 {
15     for(pos = 0; pos < 180; pos += 1) // De 0 grau a 180 em passo de 1 grau
16     {
17         meu_servo.write(pos); // Move meu_servo para a posição "pos"
18         delay(50); // Agurade 50ms para para o meu_servo atingir a posição
19     }
20 }
```

O servomotor necessita de uma biblioteca específica para o seu funcionamento. É uma biblioteca do tipo padrão e, por isso, ela precisa ser importada para o sketch. Essa importação foi feita na linha 3 da Figura 12. Os servomotores são tratados como objetos; observe que, na linha 5, o servomotor recebe o nome de “meu_servo”. A nomeação é importante pois o projeto pode utilizar mais de um servomotor.

O comando `attach()` é utilizado para determinar em qual porta digital o servomotor foi conectado. Repare na linha 10 que a sintaxe é “nome_do_servo.attach(porta)”. No nosso exemplo da Figura 12, o servomotor foi conectado à porta digital 11.

Na execução do programa, há um laço de repetição inicializando na posição 0, sendo incrementado de 1 em 1 grau até a posição 180 graus. Na linha 17, o comando “meu_servo.write(pos)” é o responsável por posicionar o servomotor. Observe a sintaxe: “objeto.write(grau)”. No exemplo, o objeto foi chamado de “meu_servo” e a variável “pos” determina o grau em que o servomotor deve ser posicionado.

O projeto vai executar um movimento iniciando em 0 grau, aumentando de grau em grau até 180 no tempo de 50 ms cada grau.

TEMA 5 — PROJETO MOTOR DE PASSO

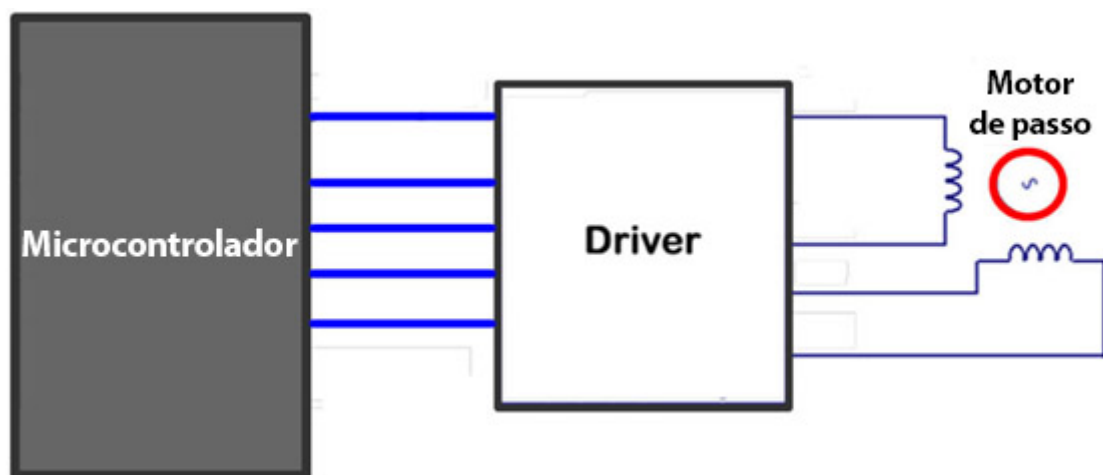
O motor de passo converte os impulsos elétricos em pequenos movimentos mecânicos, sendo necessária uma sequência correta de impulsos elétricos para que o eixo do motor possa girar. A maior vantagem desse tipo de motor é a sua precisão em um sistema de circuito aberto. Segundo Thomsen (2013), o

controle de circuito aberto significa que nenhuma informação de feedback sobre a posição é necessária. Este tipo de controle elimina a necessidade de caros sistemas de detecção, tais como codificadores ópticos. Sua posição é conhecida simplesmente através do controle dos impulsos de entrada.

O simulador Tinkercad, que foi utilizado para montar os circuitos, não tem o motor de passo como elemento. Por isso esse circuito em específico ficará fora do padrão.

O motor de passo precisa de um driver para funcionar, como vemos na Figura 13.

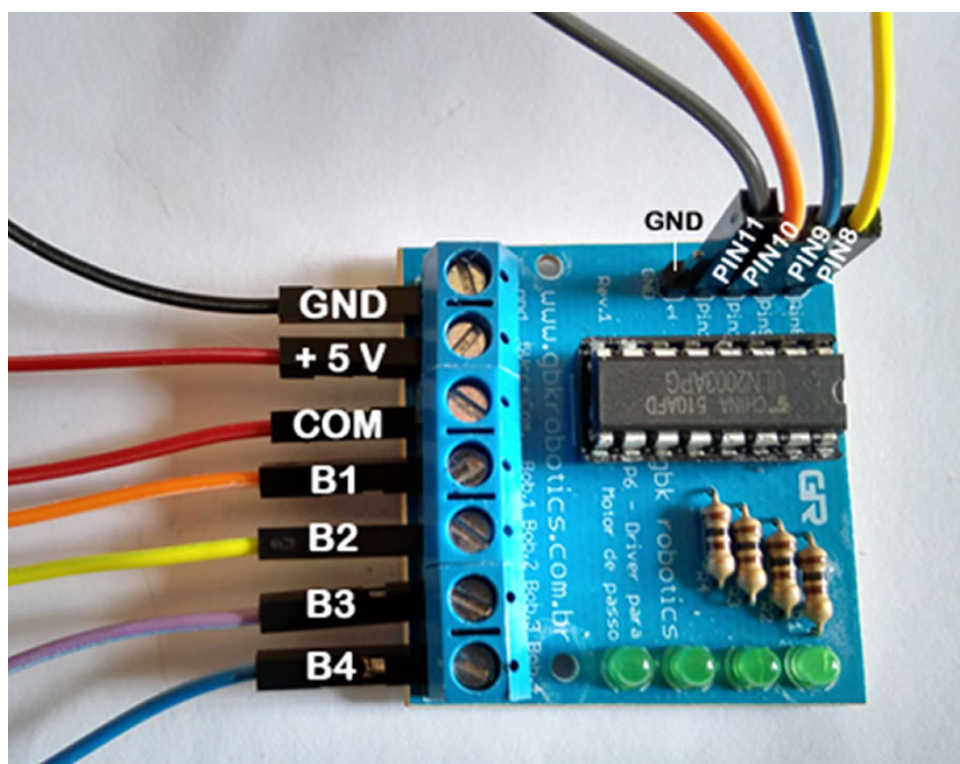
Figura 13 — Representação de um esquema com motor de passo



Fonte: Simões, 2018.

O microcontrolador representado na Figura 13 é o Arduino, e o driver é responsável pela alimentação do motor. O driver mais utilizado é o módulo ULN2003A, que possui uma matriz com sete canais (como se houvesse sete transistores TIP120 em um mesmo pacote), suportando até 500 mA por canal. Observe a Figura 14.

Figura 14 — Módulo ULN2003A

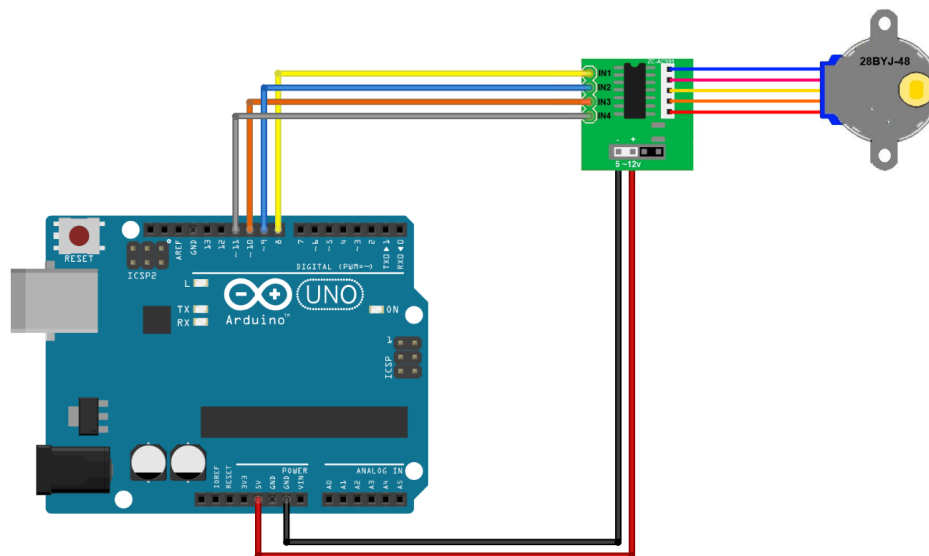


Fonte: Simão, 2018

O módulo ULN2003A tem hardware aberto e, por isso, pode ser encontrado em outros modelos, pois são fabricados por empresas diferentes, mas eles sempre apresentarão as mesmas ligações.

Na Figura 14, vemos, na parte superior, a conexão com o Arduino. Nesse exemplo, as portas digitais utilizadas são a 8, a 9, a 10 e a 11. O pino GND e o +5V são ligados ao GND e à alimentação do Arduino, respectivamente. Os pinos COM (alimentação), B1, B2, B3 e B4, são ligados ao motor. Observe a ligação na Figura 15.

Figura 15 — Representação do Projeto com motor de passo



Fonte: Thomsen, 2013.

A Figura 15 mostra uma visão macro do hardware do projeto. Nela, é possível identificar as ligações dos pinos digitais 8, 9, 10 e 11 no Arduino. Repare que todos os jumpers são conectados ao módulo ULN2003A. Após a montagem do hardware, é preciso programar.

Figura 16 — Representação do sketch do Projeto motor de passo

```
1  #include <Stepper.h>
2
3  const int stepsPerRevolution = 500;
4
5  //Inicializa a biblioteca utilizando as portas de 8 a 11 para
6  //ligacao ao motor
7  Stepper myStepper(stepsPerRevolution, 8,10,9,11);
8
9  void setup()
10 {
11   //Determina a velocidade inicial do motor
12   myStepper.setSpeed(60);
13 }
14
15 void loop()
16 {
17   //Gira o motor no sentido horario a 90 graus
18   for (int i = 0; i<=3; i++)
19   {
20     myStepper.step(-512);
21     delay(2000);
22   }
23
24   //Gira o motor no sentido anti-horario a 120 graus
25   for (int i = 0; i<=2; i++)
26   {
27     myStepper.step(682);
28     delay(2000);
29   }
30 }
```

Fonte: Adaptada de Thomsen, 2013.

Na Figura 16, vemos que, na linha 1, foi incluída a biblioteca padrão “Stepper.h”, que contém as funções necessárias para a utilização de motores de passo. Na linha 3, a constante “stepsPerRevolution” recebe o número de passos que o motor deverá dar por volta. Na linha 7, foi nomeado o motor como “myStepper” e definidas as portas necessárias para a comunicação com o Arduino. A função “setSpeed”, da linha 12, determina a velocidade inicial com que o motor (“myStepper”) deve girar como 60. Dentro da execução do programa, existem dois laços de repetição, cujos valores são passados por parâmetros na função step, conforme a Tabela 1.

Tabela 1 — Tabela graus x passos

Para girar ... graus,	serão necessários ... passos	Passos (Valor inteiro arredondado)
30	170,67	171
60	341,33	341
90	512,00	512
120	682,67	683
180	1024,00	1024

Fonte: Simão, 2018.

A Tabela 1 mostra o valor que deve ser passado na função “step” para o motor girar para um determinado grau. Note que, na linha 20 da Figura 16, o valor passado foi -512, fazendo o eixo girar 90 graus no sentido horário. Já na linha 27, foi passado o valor 682, fazendo o eixo girar 120 graus no sentido anti-horário.

Lembre-se de que é possível movimentar o eixo para qualquer grau, basta informar o seu valor correspondente na função “step”.

REFERÊNCIAS

REIS, F. dos. **Como funciona um potenciômetro**. Disponível em: <<http://www.bosontreinamentos.com.br/eletronica/curso-de-eletronica/como-funciona-um-potenciometro/>>. Acesso em: 27 mar. 2019.

STRAUB, M. G. **Braço robótico para Arduino**. Disponível em: <<https://blog.usinainfo.com.br/braco-robotico-para-arduino/>>. Acesso em: 29 mar. 2019.

THOMSEN, A. **Controlando um motor de passo 5V com Arduino**. Disponível em: <<https://www.filipeflop.com/blog/controlando-um-motor-de-passo-5v-com-arduino/>>. Acesso em: 30 mar. 2019.

SIMÕES, H. M. C. **Controlando um motor de passo com Arduino**. Disponível em: <<http://blog.moduloeletronica.com.br/controlando-um-motor-de-passo-com-o-arduino/>>. Acesso em: 30 mar. 2019